

《人工智能开发技术》

实验报告

专　　业：计算机科学与技术
班　　级：24 计科 4 班
学　　号：2410311411
姓　　名：何嘉欣
实验（五）：Transformer 与大语言模型推理基础
实验日期：2026.10.26

《人工智能开发技术》课程实验报告

实验名称	Transformer 与大语言模型推理基础					实验序号	五
姓　　名	何嘉欣	专　　业	计科	班　　级	24 计科 4	学　　号	2410311411
实验类型	综合型	实验日期	2026.10.26	指导教师	陈文静	成　　绩	

仓库地址: https://github.com/wxzn222/AID-homework

一、实验目的

1. 实验目标

本实验在 CPU 上加载 Qwen2.5-0.5B-Instruct，观察 Transformer 自回归推理的完整链路：文本分词、聊天模板、生成提示、前向计算、下一 token 采样与解码。实验不把模型输出当作天然可信对象，还要用 token 预算和 Pydantic 契约把“能生成”提升为“可审计、可拒绝”。

具体目标包括：理解 token 与字符不一一对应；区分贪心、低温和高温采样；在有限上下文中保留 system 与最新问题；从带解释或代码围栏的文本中提取 JSON，并校验字段类型、范围、唯一性和跨字段业务规则；明确本地模型与在线 API 的共同边界。

2. 完成内容

- 完成 FILL-01～FILL-14，覆盖随机性与 CPU 线程、模型加载、只解码新增 token、token 片段、聊天模板、三种解码、上下文裁剪、JSON 扫描、Pydantic 字段与业务校验。
- 真实加载 494,032,768 参数模型，记录缓存、设备、加载时间、峰值内存和三种解码输出。
- 完成上下文 160 token 预算实验、四类结构化反例和无配置在线接口安全跳过，并保留带学号的两张验收截图。

二、Transformer 推理流程

1. 从文本到回答的链路

推理链路可以概括为：原始文本 → tokenizer 切分并映射为 token ID → 套用角色聊天模板 → 在末尾追加 assistant 生成提示 → Transformer 前向计算得到下一个 token 的 logits → 按贪心或采样策略选 token → 将新 token 追加到序列并重复，直到 EOS 或 max_new_tokens。

```
text → input_ids → chat template → model.generate
    → logits → next token → append → detokenize
```

Qwen 是因果语言模型，每个位置只能看到自己和之前的 token。自注意力把当前表示与历史位置对齐：

$$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}(QK^{\top} / \sqrt{d_k})\, V$$

本实验使用 float32 CPU 推理，并在 torch.inference_mode() 下关闭梯度记录。model.generate 返回的是“完整提示 + 新生成 token”，因此必须记住 prompt_length，只解码 output_ids[0, prompt_length:]，否则回答会重复 system 和 user 原文。

2. 为什么需要聊天模板

指令模型训练时看到的不是裸文本，而是 system、user、assistant 三种角色及其控制标记。聊天模板负责把这些角色边界渲染成模型熟悉的特殊 token；add_generation_prompt=True 再追加 assistant 起始标记，告诉模型下一步应当由助手续写。模板本身也占上下文预算，所以不能用字符数估算。

3. 可审计的生成边界

生成文本只代表“模型输出了这些字符”，不代表字段一定齐全、类型一定正确或内容一定真实。实验将生成、提取、模式校验和业务判断拆开：模型负责候选文本，应用负责接受或拒绝。这样即使模型返回了带 Markdown 围栏的 JSON，也必须经 extract_json_object 与 StudyPlan 校验后才能进入业务对象。

三、环境、CPU、依赖、本地模型和缓存记录

1. 软件与硬件环境

实验使用 Windows CPU 环境、Python 3.12.13 和项目独立虚拟环境 .venv；torch 为 2.13.0+cpu，transformers 5.15.1，pydantic 2.13.4，openai 3.3.1，psutil 7.2.2。torch_cuda_available=False，selected_device=cpu，固定计算线程 4，机器物理内存 27.8 GiB。python -m pip check 返回 No broken requirements found.。

表 1　环境与依赖记录

项目	真实值	说明
Python	3.12.13	独立 .venv
PyTorch	2.13.0+cpu	CPU wheel，无 CUDA 依赖
Transformers	5.15.1	本地模型加载与生成
Pydantic	2.13.4	结构化字段与业务规则
OpenAI	3.3.1	可选在线接口客户端
设备/线程	cpu / 4	float32 推理
物理内存	27.8 GiB	psutil 实测

```
PS> .venv\Scripts\python.exe .\00_env_check.py
python=3.12.13
torch=2.13.0+cpu
transformers=5.15.1
pydantic=2.13.4
openai=3.3.1
psutil=7.2.2
torch_cuda_available=False
selected_device=cpu
cpu_threads=4
physical_memory_gib=27.8
model_id=Qwen/Qwen2.5-0.5B-Instruct
PS> .venv\Scripts\python.exe .\student_identity.py --show --stage "ENV_CHECK"
STUDENT_ID=2410311411
EXPERIMENT_ID=05
STAGE=ENV_CHECK
IDENTITY_CODE=5EDE7DD6D914
REGISTERED_AT=2026-10-26T14:50:10+08:00
CAPTURED_AT=2026-10-26T14:56:03+08:00
PS> .venv\Scripts\python.exe -m pip check
No broken requirements found.
```

图 1　环境、依赖、身份与 pip 检查（学号 2410311411，阶段 ENV_CHECK）

2. 本地模型与缓存

模型 ID 固定为 Qwen/Qwen2.5-0.5B-Instruct，缓存目录为实验目录下的 hf_cache；模型类为 Qwen2ForCausalLM，参数量 494032768，设备为 CPU，声明上下文上限 32768，pad_token_id=151643。完整缓存约 953.4 MiB，包含权重、tokenizer 与配置。

表 2　模型加载与资源记录

指标	首次加载	缓存命中	说明
加载时间	71.973 s	2.906 s	首次包含下载/写缓存
峰值 RSS	3238.4 MB	3214.9 MB	含解释器、库和权重
缓存大小	953.4 MiB	953.4 MiB	hf_cache 全部文件
参数量	494,032,768	494,032,768	0.5B 教学模型
上下文上限	32768	32768	模型配置值

首次下载加载耗时明显高于缓存命中，说明实验性能记录必须注明数据状态。峰值 RSS 约 3.2 GB，大于权重文件本身，因为还包括 Python、PyTorch、临时张量和推理运行时。

四、token、token ID、聊天模板与生成提示后缀观察

1. 文本 token 观察

使用 add_special_tokens=False 编码三个纯文本，记录 ID 与 token 片段。结果如下：

- 人工智能：4 个字符 → 1 个 token，ID=[104455]；常见中文短语被合并。
- 人工智能开发技术：8 个字符 → 3 个 token，ID=[104455, 100013, 99361]；不是字符数的一半。
- AI：2 个字符 → 1 个 token，ID=[15469]；英文缩写同样是整体 token。

人工智能开发技术只比人工智能多“开发技术”四个汉字，却从 1 个 token 变为 3 个 token；字符数从 4 到 8 是 2 倍，token 数却是 3 倍。tokenizer 使用数据驱动的合并规则，常见片段可能整体成词，生僻组合或跨词边界会被拆开，所以不能用字符数、汉字数或英文单词数近似 token 数。

token_report.json 中的 pieces 是 byte-level BPE 的原始片段显示，可能出现看似乱码的拉丁字符；这只是字节映射，不是解码后的用户文本。用 tokenizer.convert_tokens_to_string 可以还原为“人工智能”“人工智能开发技术”，最终面向用户仍应使用 tokenizer.decode。

2. 聊天模板与生成提示后缀

同一组 system/user 消息在不带 add_generation_prompt 时，以用户消息和结束标记收尾；带该参数后，末尾追加 assistant 起始标记。真实输出：

```
added_suffix='<|im_start|>assistant\n'
without_prompt_tail=...user\n用一句话说明注意力机制的作用。<|im_end|>\n
with_prompt_tail=...<|im_start|>assistant\n
```

角色边界的作用是告诉模型“哪些是长期规则、哪些是当前请求、从哪里开始回答”。控制 token 既参与前向计算也占据上下文预算；因此 token_count 必须对模板化后的 input_ids 计数，而不是只数用户文字。

五、贪心、低温和高温三种解码策略比较

三组策略在相同 system/user 消息、相同 32 个最大新 token 和相同随机种子下运行。greedy 使用 do_sample=False；focused 使用 temperature=0.3、top_p=0.8；diverse 使用 temperature=1.0、top_p=0.95。真实结果如下：

表 3　三种解码策略的真实输出

策略	参数	提示 token	新增 token	耗时	生成文本
greedy	do_sample=False	36	19	2.943 s	注意力机制通过分发信息到不同的层来提高模型的鲁棒性和效率。
focused	T=0.3, p=0.8	36	18	2.930 s	注意力机制用于在多任务学习中提高模型的性能和鲁棒性。
diverse	T=1.0, p=0.95	36	15	2.508 s	注意力机制用于增强网络对输入信息的专注和理解能力。

结果解释

贪心输出把注意力描述为“分发信息到不同的层”，但标准注意力更核心的是在同一序列内按相关性对 Value 做加权聚合；“鲁棒性和效率”也不是注意力的直接定义。低温输出只给出“多任务学习”的下游用途，概念不完整。高温输出“增强网络对输入信息的专注和理解能力”最接近注意力按输入相关性分配权重的含义，虽然“理解”带有拟人化色彩。

三句都流畅，但不能按流畅度判定正确性。diverse 本次更准确不代表高温总是更好：低温适合事实问答、代码和结构化输出，高温适合创意发散；无论哪种策略，都要通过外部校验或人工概念证据复核。

三种策略的新增 token 数分别为 19、18、15，均不超过 32；耗时约 2.5～2.9 秒，说明在本机 CPU 上短回答可以接受，但长文本生成会线性变慢。

六、上下文 token 预算、裁剪规则和边界用例

1. 计数与裁剪规则

token_count 用 apply_chat_template(..., tokenize=True, add_generation_prompt=True, return_dict=True) 得到真实 input_ids 长度。fit_messages 先单独保存首条 system 和最后一条最新 user，再把中间历史按 user/assistant 成对处理；每轮删除最旧两条，直到模板化 token 数不超过预算。若 system 加最新问题仍超预算，则抛出 ValueError，而不是静默按字符截断。

```
candidate = system + history + newest
while token_count(candidate) > max_tokens: history = history[2:]
system 与 newest 不参与删除；放不下时显式失败
```

2. 160 token 主实验

原始 14 条消息（6 轮历史加最新问题）模板化后为 354 token；预算 160 时裁剪到 6 条、138 token，删除 8 条。system_preserved=True、newest_preserved=True，角色仍以 system 开始、user 结束。

3. 边界用例

表 4 不同 token 预算下的裁剪边界

输入预算	裁剪后 token	保留消息	删除消息	结果
160	138	6	8	保留最近两轮历史
140	138	6	8	与 160 相同，下一对放不下
120	84	4	10	再删除一对旧历史
80	30	2	12	只剩 system 与最新问题
10	—	—	—	ValueError：最小必要消息仍超预算

这组结果表明裁剪不是“删到刚好小于上限”的字符截断：140 与 160 保留了相同对话，因为下一组历史成对加入就会超限；80 时已无历史可删，只剩 30 token 的 system + 最新问题；10 时最小必要消息都放不下，程序必须报错而不是删掉问题或系统约束。实际调用还应从模型上下文上限中扣除 max_new_tokens 和安全余量，不能用满 32768。

七、JSON 提取、Pydantic 字段约束和跨字段规则

1. 原始输出与提取

本地模型在结构化任务中生成了带 Markdown 围栏的文本，而不是裸 JSON：

```
```json
{
  "topic": "Transformer",
  "difficulty": 4,
  "keywords": ["transformer", "deep learning", "neural networks"],
  "estimated_minutes": 120
}
```
```

extract_json_object 从第一个左花括号开始，跟踪花括号深度，并维护 in_string 与 escaped 状态，忽略字符串内部和转义后的花括号；深度回到 0 时先调用 json.loads 做语法校验，再返回首个完整对象。这样模型前后附加解释或代码围栏不会直接进入 Pydantic。

2. 字段与跨字段约束

表 5　StudyPlan 字段与业务规则

字段/规则	约束	作用
topic	str，2～30 字符	防止空主题或超长主题
difficulty	int，1～5	拒绝字符串与越界难度
keywords	list[str]，2～4 项	限制关键词数量
keywords 清洗	strip、丢弃空串、拒绝重复	统一规范并发现重复
estimated_minutes	int，10～120	限制学习时长
跨字段规则	difficulty≥4 时 minutes≥30	拒绝高难度短时间的不合理计划

本次原始输出提取后，StudyPlan 校验为 PASS，规范对象为 topic=Transformer、difficulty=4、keywords=[transformer, deep learning, neural networks]、estimated_minutes=120。难度 4 且时间 120 满足“至少 30 分钟”的业务约束。

```
validation=PASS
{"topic":"Transformer","difficulty":4,
 "keywords":["transformer","deep learning","neural networks"],
 "estimated_minutes":120}
```

structured_result.json 同时保存 raw 与 validated：raw 保留代码围栏和模型原始排版，validated 保存清洗后的规范对象。这样既能证明提取确实发生，也不会把模型原文误当成已接受业务数据。

八、4 类故意反例与拒绝原因

用同一 StudyPlan 对四类故意反例执行 model_validate_json，全部被拒绝，没有出现 UNEXPECTED_PASS。

表 6　四类反例的拒绝证据

反例	输入问题	Pydantic 错误类型	字段位置	拒绝原因
wrong_type	difficulty="困难"	int_parsing	difficulty	类型不是整数
out_of_range	difficulty=8	less_than_equal	difficulty	超过 5
duplicate_keywords	keywords=[Token, Token]	value_error	keywords	清洗后重复
business_rule	difficulty=5, minutes=20	value_error	()	高难度少于 30 分钟

错误位置表明问题发生在字段解析、范围约束、字段清洗还是模型级业务规则；报告不应把四行 REJECTED 简化成一句"校验通过"，而应保留具体 type 与 loc。

```
PS> .venv\Scripts\python.exe .\05_validation_cases.py
wrong_type=REJECTED type=int_parsing location=('difficulty',)
out_of_range=REJECTED type=less_than_equal location=('difficulty',)
duplicate_keywords=REJECTED type=value_error location=('keywords',)
business_rule=REJECTED type=value_error location=()
PS> .venv\Scripts\python.exe .\student_identity.py --show --stage "STRUCTURED_VALIDATION"
STUDENT_ID=2410311411
EXPERIMENT_ID=05
STAGE=STRUCTURED_VALIDATION
IDENTITY_CODE=5EDE7DD6D914
REGISTERED_AT=2026-10-26T14:50:10+08:00
CAPTURED_AT=2026-10-26T14:59:12+08:00
```

图 2　四类结构化反例与身份验收（全部 REJECTED，阶段 STRUCTURED_VALIDATION）

九、在线接口状态、资源限制

1. 在线接口状态

当前会话未设置 LLM_API_KEY、LLM_BASE_URL、LLM_MODEL，05_online_api.py 在发起请求前识别到三个变量缺失，输出：

```
SKIP online API: missing LLM_API_KEY, LLM_BASE_URL, LLM_MODEL
```

进程退出码为 0，未发出网络请求，也未伪造 validation=PASS。因此本报告的在线接口结论是 SKIP，而不是在线成功或在线失败；本地模型、token、解码、上下文和结构化结果不受影响。

2. 资源与能力边界

- 模型参数量：494,032,768（0.5B 教学规模）。
- 缓存：953.4 MiB，包含权重、tokenizer 与配置。
- 首次加载：71.973 s，峰值 RSS 3238.4 MB（含下载与缓存写入）。
- 缓存命中加载：2.906 s，峰值 RSS 3214.9 MB。

- 短句生成：2.508～3.044 s，新增 token 分别为 19、18、15。
- 声明上下文上限 32768 token；固定 CPU 线程 4。

0.5B 模型的主要优势是无需 GPU、缓存约 1 GB、缓存命中后数秒即可加载；主要代价是知识容量和推理深度有限，短句都可能出现概念偏差。RSS 包含解释器、依赖库、权重与临时张量，不能用它直接等同于模型权重大小；不同机器、线程数和版本也会改变耗时。

在线路径复用同一套 StudyPlan 和 extract_json_object，只替换生成提供方。工程上必须把 HTTP 状态、响应解析、模式校验和业务判断分层：HTTP 200 只说明请求成功，不代表内容可接受。

十、分析题和总结

1. 实验手册第 12 节分析题

1. 人工智能与人工智能开发技术的字符数和 token 数为什么不成比例？分词器使用数据驱动的 BPE 合并规则，常见片段可能整体成为一个 token。人工智能 4 个字符被合并为 1 个 token（ID 104455）；人工智能开发技术 8 个字符被切成 3 个 token（104455、100013、99361）。字符数增加 2 倍，token 数却增加 3 倍；AI 两个字符也是 1 个 token。token 数取决于词表和合并规则，不是字符数或汉字数的线性函数。

2. 聊天模板中的 system、user、assistant 边界对模型有什么作用？system 提供持续生效的角色与规则，user 表示当前请求，assistant 标记模型应答的起点和结束。模板把这些边界渲染为特殊控制 token，让指令模型区分长期约束、输入和输出。add_generation_prompt=True 在末尾追加 <|im_start|>assistant，提示模型从这里开始续写；本次 added_suffix 正是该字符串。

3. 如果解码 output_ids[0] 而不切掉提示，会看到什么？会看到完整提示和回答的拼接，包括 system/user 原文、角色边界与结束标记。model.generate 返回提示 token 加新增 token，output_ids[0] 是整条序列；直接 decode 会让回答重复问题和系统消息，也不便于统计新增 token。因此先记录 prompt_length，再解码 output_ids[0, prompt_length:]。

4. 为什么贪心解码无需 Temperature？低温与高温采样各适合什么场景？贪心每步直接取 logits 最大的 token，不做概率采样，Temperature 不参与 argmax，因此配置中没有 temperature。低温配合较小 top_p 会压缩候选、集中概率，适合事实问答、代码和 JSON；高温和更大 top_p 增加多样性，适合创意写作或头脑风暴，但也更容易偏离约束。本次 focused 为 T=0.3、p=0.8，diverse 为 T=1.0、p=0.95。

5. 固定随机种子能控制哪些差异，不能控制哪些差异？固定 Python/Torch 种子可以控制采样随机数序列和同一环境下重复运行的可复现性；本次三种策略每轮前重置 SEED=2026。它不能保证不同 Torch/Transformers 版本、不同 CPU 数学库、不同线程调度、不同 tokenizer 文件或不同硬件上的逐 token 完全一致，也不能消除模型本身的知识错误。

6. 三种输出中哪一句最准确？请逐句给出概念证据，而不是按流畅度判断。diverse 的“注意力机制用于增强网络对输入信息的专注和理解能力”最接近注意力的核心：按输入位置相关性分配权重、聚合信息。greedy 说“分发信息到不同的层”把层间传递与序列内注意力混在一起，且“鲁棒性和效率”不是定义。focused 只提到多任务学习的下游用途，没有解释 Q/K/V 或加权聚合。三句都流畅，但概念完整性不同。

7. 为什么上下文裁剪必须给回答预留 token，而不是用满模型上限？模型上下文上限同时容纳提示和后续生成；如果提示占满 32768，就没有位置生成回答，KV 缓存和内存也会更紧张。实际提示预算应为上下文上限减去 max_new_tokens 和安全余量，并考虑模板控制 token、系统消息与最新问题。本实验在 160 token 预算下先裁剪历史，正是把有限窗口留给当前问题和回答。

8. 删除单条历史消息可能破坏什么角色结构？对话通常按 user/assistant 成对组织；只删一条会留下没有问题的助手回答、连续 user 或悬空的 assistant 历史，模板中的角色交替和语义依赖被破坏。因此 fit_messages 每次从 history 删除两条，保留 system 和最新 user；删除消息数必须为偶数。

9. JSON 语法正确但 Pydantic 仍拒绝的案例有哪些？本次有四类：difficulty="困难" 触发 int_parsing；difficulty=8 触发 less_than_equal；keywords=[Token, Token] 在字段校验器中触发 value_error；difficulty=5、estimated_minutes=20 通过单字段但在模型级业务规则触发 value_error。另有 extra="forbid" 可拒绝模型臆造出的额外字段。

10. extra="forbid" 如何帮助发现模型幻觉出的字段？若模型输出 priority、resources 等 schema 未定义的字段，默认宽松模型可能会静默忽略，导致隐藏语义丢失且难以发现。extra="forbid" 会把这些字段视为验证错误，错误位置指出额外字段；应用必须明确决定是否扩展契约，而不是默默接受模型自创结构。

11. 为什么原始文本和验证对象应同时保存？raw 保留模型实际输出、代码围栏和排版，用于审计、复现和比较模型行为；validated 保存清洗与校验后的规范对象，供业务使用。只保存 raw 会把未校验文本带入下游，只保存 validated 又无法知道模型原始表达是否被改写。本次 structured_result.json 同时保存两者。

12. 本地与在线路径共用模式校验有什么工程价值？同一 StudyPlan、同一提取器和同一校验器让本地模型与在线服务在同一契约下比较，切换供应商时不必重写业务规则；测试、错误定位和安全边界集中，避免某个提供方的宽松输出绕过字段和业务约束。在线接口只是生成层替换，接受标准不变。

13. 若 API 返回 200 但 validation=FAIL，这次应用调用算成功吗？说明分层判定。不算应用任务成功。HTTP 200 只表示传输层请求被服务接收并返回；还需要响应可解析、能提取 JSON、通过 Pydantic 字段与跨字段规则，才形成可用业务对象。正确表述是"网络请求成功，但结构化输出失败"。本次在线路径因缺少配置为 SKIP，没有发出请求，也没有把任何在线结果记为成功。

14. 0.5B 模型的性能优势与质量代价分别是什么？优势是 494,032,768 参数、缓存约 953.4 MiB、CPU float32 可运行；首次加载 71.973 秒，缓存命中 2.906 秒，短句生成约 2.5～3.0 秒，适合课堂和低资源演示。代价是知识容量与推理深度有限，本次注意力定义就出现概念偏差；复杂事实、长链推理和严格 JSON 都可能需要更大模型或外部校验。

2. 实验总结

本实验把 Transformer 推理拆成可观察的工程环节：token 与 ID、聊天模板、生成提示、只解码新增 token、三种采样、上下文裁剪、JSON 提取和 Pydantic 校验。真实运行表明本地模型可以完成短回答和结构化生成，但不能因为句子流畅就跳过校验。

本次结构化原始输出带代码围栏，提取器仍正确得到首个 JSON 对象并通过 StudyPlan；四类反例全部按预期拒绝。上下文从 354 token / 14 条裁剪到 138 token / 6 条，system 与最新问题均保留；预算 10 时显式报错，没有静默删除关键消息。在线接口为 SKIP，报告不冒充在线成功。

3. 局限与改进方向

- 0.5B 模型适合演示但概念精度有限；可比较更大本地模型或经授权的在线模型，但必须保持同一时间、同一提示和同一校验器。
- 上下文裁剪目前只按成对删除最旧历史，没有摘要、检索或重要性评分；可加入滚动摘要并保留关键事实。
- 结构化生成仍可能出现字段值不理想或额外字段；可增加修复提示、重试预算和更细的业务规则，但不能手工篡改 raw。
- 本次在线路径为 SKIP，未进行跨服务质量和延迟比较；后续应在合法配置下记录 401/404/429/超时与 validation 分层状态。

4. 复现命令与产物

```
.\.venv\Scripts\python.exe .\00_env_check.py
.\.venv\Scripts\python.exe .\00_model_check.py
.\.venv\Scripts\python.exe .\01_inspect_tokens.py
.\.venv\Scripts\python.exe .\02_compare_decoding.py
.\.venv\Scripts\python.exe .\03_context_window.py
.\.venv\Scripts\python.exe .\context_boundary.py
.\.venv\Scripts\python.exe .\04_structured_json.py
.\.venv\Scripts\python.exe .\05_validation_cases.py
.\.venv\Scripts\python.exe .\05_online_api.py
```

产物位于 outputs：token_report.json 保存三组 token ID 与片段；decoding_results.json 保存三种策略的文本、耗时、提示/新增 token 和峰值内存；structured_result.json 同时保存模型 raw 与 validated 对象；student_identity.json 记录学号 2410311411、实验编号 05、身份校验码 5EDE7DD6D914。